Advanced Practice Progress Review 2

Please discuss and complete this with your academic supervisor, and upload it to Blackboard, using the Assessment area (Progress Review 2 Submission) by the end of week 9.

Use the Progress Review to help you identify what things are going well, what you can improve and continue the reflective analysis ready for writing your final assessment for Advanced Practice. You can look back at both Progress Reviews to help you write your assessment.

Please save the file with your name appended (e.g. Progress Review 2_Rushin-Chape).

Student Name: Damilola Adebayo

Supervisor Name: Dr Xinjun Cui

Date: 22/03/2024

To be completed by the student

Summary of Tasks since your last review

Please summarise the tasks set for your internship since your last review

In my previous review, I outlined the primary focus of my research internship, which revolves around Data Science and machine learning within the realm of Granular Flow Simulations. I am pleased to report that I have continued to pursue this topic diligently and have made substantial progress since then.

Notable achievements include the development of various programmable mathematical functions using Python and the adept analysis of data employing frameworks previously mentioned in my initial review. The main objectives of this internship, established from the outset, remain unchanged and include:

1. Attaining a comprehensive understanding of granular flow.
2. Collecting and processing diverse data from experiments and research related to granular flow.
3. Acquiring proficiency in Data Analytics and Machine Learning tools.
4. Applying aforementioned tools for feature encoding and imputation purposes.
5. Constructing a Machine Learning model to predict the position of individual particles or sets of particles at specific time steps.
6. Enhancing comprehension of the mathematical underpinnings of the machine learning model.

Objectives Achieved so far

I am pleased to announce that I have successfully accomplished all the milestones outlined in my previous review. This signifies the achievement of the first five objectives, with the only remaining task being the continual refinement and optimization of my machine learning model to enhance its predictive accuracy.

Objective in Progress

The sole objective that remains to be fulfilled is to augment my understanding of the mathematical principles underlying machine learning. This task is currently underway and is progressing steadily.

Describe what has gone well since your last review

Since my last review, significant strides have been made in my work area, marked by substantial improvements and accomplishments. I attribute these advancements to the invaluable support and guidance provided by my supervisor, Prof. Xinjun. The progress achieved encompasses various domains, ranging from enhanced proficiency in Python programming to the practical application of this knowledge in data analytics utilizing datasets from platforms such as Kaggle. Additionally, I have attained a robust understanding of Machine Learning, as evidenced by my successful development of high-accuracy machine learning models across diverse datasets. Finally, I was able to build the machine learning model for my research on granular flow data having gathered lots of data.

Advancements in Python Programming

Noteworthy progress has been made in Python programming, culminating in the creation of several applications, some of which are details of which are detailed below. These applications are for practical purposes and can be executed via terminal. Specifically:

1. **Quadratic Equations Solver:** I developed a solver capable of handling quadratic equations while accounting for potential errors such as ValueError, particularly when dealing with the square root of negative numbers. (*see* Figure 1: Quadratic Equation Solver)

Figure 1: Quadratic Equation Solver

2. **Maximum Number Finder:** An application designed to return the maximum number within a given list.
3. **Pound to Kilogram Converter:** This application facilitates weight conversion and issues a warning when an invalid unit is entered, aside from 'L' for pounds or 'K' for kilograms. (*see* Figure 2: Weight Converter App)

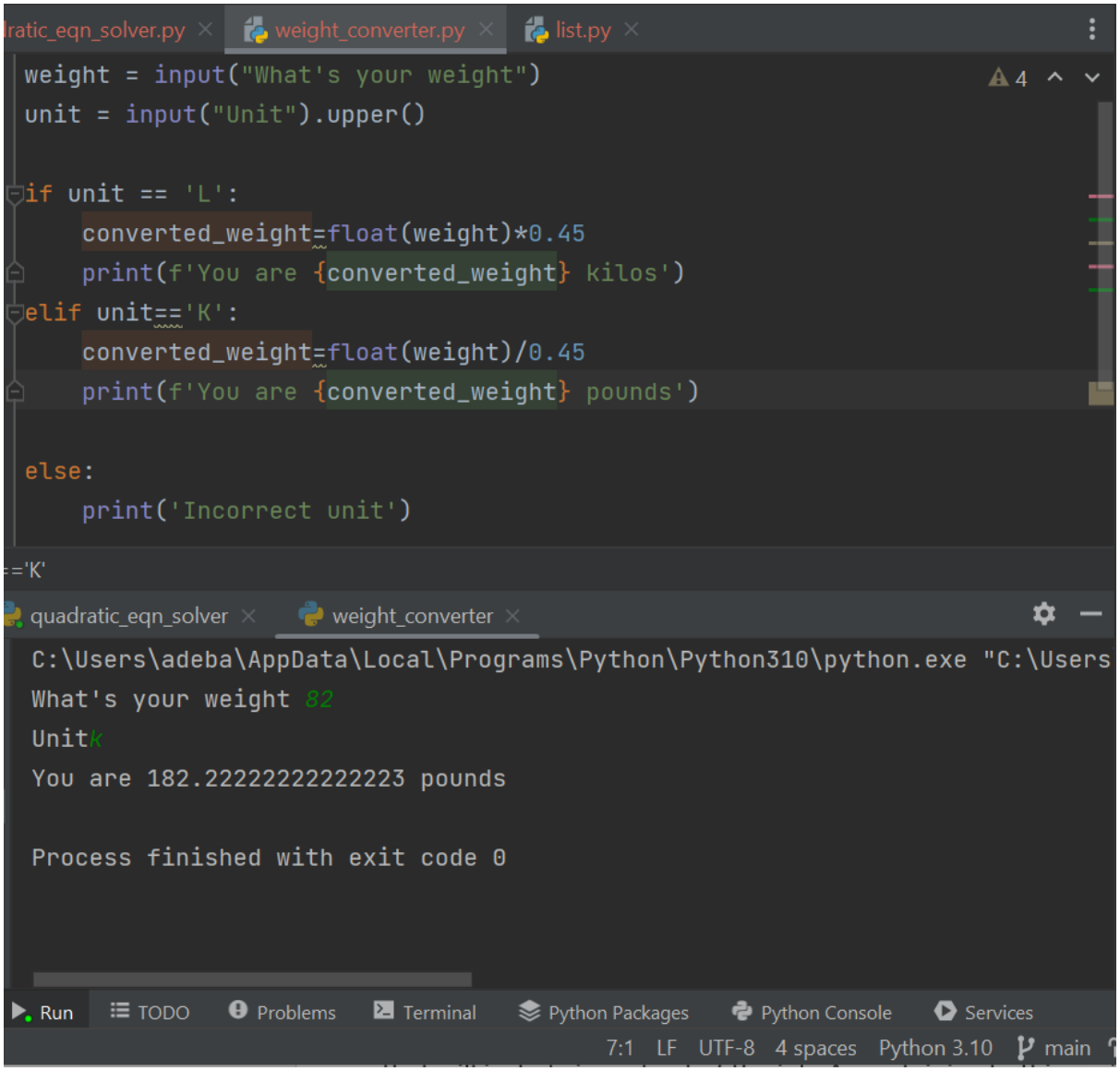


Figure 2: Weight converter app (kg to lb and vice versa) showing a sample conversion of 82kg to 182.2 pounds

Proceeding to **Data Analytics,**

Data analytics have involve a comprehensive understanding of mathematical and statistical theories, coupled with proficiency in robust frameworks such as Pandas and NumPy. These tools are instrumental in analyzing large-scale datasets, characteristic of the project's scope. Samples of results derived from real-world problem analysis are provided in the attached documentation.

Building Machine Learning Models

A notable accomplishment includes the successful construction and development of machine learning models tailored to various real life data sets found on Kaggle platform.

Analysing and Building the Machine Learning Model for the data space in Granular Flow Simulations

This is my research focus and I have hitherto completed it. The online version of the code solutions can be accessed via the provided links.

Github Link:

Additionally, an offline copy of the codes is included in the appendix, mirroring the online version in its entirety.

Describe what you feel you need to continue to work on to improve your skills/professionalism etc.

As the adage goes, learning is an ongoing journey, and I recognize the need to continuously enhance my skill set, particularly in the realm of machine learning, both within and outside of my current role. Regarding this project, my primary focus is on refining and optimizing the existing models to achieve greater accuracy. I intend to dedicate the next three weeks of spring break specifically to this endeavor.

Furthermore, I aspire to expand my expertise into imagery recognition, which represents another vital aspect of machine learning. By delving into this area, I aim to broaden my skill set and deepen my understanding of the diverse applications of machine learning algorithms.

In addition to these immediate goals, I harbor a long-term ambition of developing my own machine learning package. This envisioned program would surpass existing models by delivering highly accurate predictions, approaching a score of 100%. This aspiration aligns with my future aspirations for pursuing a PhD, where such a project could serve as a cornerstone of my research endeavours.

Lastly, I intend to pursue professional certification in the field to fortify my engineering aspirations. I firmly believe that proficiency in data analysis is integral to effective decision-making, a hallmark of great engineering. By attaining certification, I aim to bolster my credentials and underscore my commitment to excellence in this domain.

To be completed by the academic supervisor

In your opinion, is the student meeting your expectations?

Choose an item.

Please provide some formative feedback to identify good practice and help the student continue improving

Click or tap here to enter text.

Granular_Flow

March 25, 2024

1 Data Analytics and Machine Learning of the Data space in Granular Flow

[1]:
```
%matplotlib inline
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

[2]:
```
#pd.set_option('display.max_column', None)
#pd.set_option('display.max_rows', None)
```

2 1.0 Combining the data gathered for each time step into a single csv file

[3]:
```
granular_flow_data_paths=['./Tables/XYZ_Internal_Table_table_100.csv','./Tables/
↪XYZ_Internal_Table_table_200.csv', './Tables/XYZ_Internal_Table_table_300.
↪csv', './Tables/XYZ_Internal_Table_table_400.csv', './Tables/
↪XYZ_Internal_Table_table_500.csv', './Tables/XYZ_Internal_Table_table_600.
↪csv', './Tables/XYZ_Internal_Table_table_700.csv', './Tables/
↪XYZ_Internal_Table_table_800.csv', './Tables/XYZ_Internal_Table_table_900.
↪csv', './Tables/XYZ_Internal_Table_table_1000.csv', './Tables/
↪XYZ_Internal_Table_table_1100.csv' ]

granular_flow_dataset = []
for datapath in granular_flow_data_paths:
    df=pd.read_csv(datapath)
    granular_flow_dataset.append(df)
```

[4]:
```
y=pd.concat(granular_flow_dataset)
Combined_flow_data=y.to_csv('./Combined flow data.csv')
Combined_flow_data
```

3 2.0 Working With the Saved file

3.1 2.1 Reading (Loading) the combined saved data

[5]:
```
Granular_flow_data= pd.read_csv('./Combined flow data.csv')
Granular_flow_data
```

[5]:
```
        Unnamed: 0  Time (s)  Parcel Centroid[X] (m)  Parcel Centroid[Y] (m)  \
0                0      0.02               -0.105754                0.031774
1                1      0.02               -0.106013                0.024868
2                2      0.02               -0.106014                0.027605
3                3      0.02               -0.106013                0.024352
4                4      0.02               -0.106007                0.026172
..             ..       ..                     ..                     ..
132311       18270      0.22               -0.117250                0.040575
132312       18271      0.22               -0.117254                0.040656
132313       18272      0.22               -0.117258                0.031591
132314       18273      0.22               -0.117265                0.035741
132315       18274      0.22               -0.117269                0.025780

        Parcel Centroid[Z] (m)  Particle Velocity[i] (m/s)  \
0                     0.002455                    0.761367
1                     0.015521                    0.648313
2                     0.006328                    0.647975
3                     0.008947                    0.648059
4                     0.016868                    0.648702
..                       ..                         ..
132311                0.014844                    0.538821
132312                0.010430                    0.538780
132313                0.021938                    0.538739
132314                0.021986                    0.538658
132315                0.004942                    0.538617

        Particle Velocity[j] (m/s)  Particle Velocity[k] (m/s)  \
0                     3.673675e-03                   -0.138890
1                     2.494714e-04                   -0.251882
2                    -1.013336e-04                   -0.251266
3                     4.421311e-05                   -0.251678
4                     7.440611e-04                   -0.250163
..                             ..                       ..
132311                1.922194e-07                   -0.108071
132312                3.806327e-09                   -0.108018
132313               -2.009805e-07                   -0.107967
132314                9.289315e-08                   -0.107863
132315               -2.137597e-08                   -0.107811

        Particle Velocity: Magnitude (m/s)  Volume Fraction of Phase 1  \
0                                 0.773941                     0.064024
```

```
1                                0.695525                     0.077178
2                                0.694986                     0.064024
3                                0.695214                     0.064024
4                                0.695267                     0.077178
..                                  ...                          ...
132311                           0.549552                     0.040602
132312                           0.549502                     0.086906
132313                           0.549452                     0.048327
132314                           0.549351                     0.040602
132315                           0.549301                     0.160566

        Wall Shear Stress: Magnitude (Pa)  Report: Element Count 1  \
0                                       0                      2777
1                                       0                      2777
2                                       0                      2777
3                                       0                      2777
4                                       0                      2777
..                                    ...                       ...
132311                                  0                     18275
132312                                  0                     18275
132313                                  0                     18275
132314                                  0                     18275
132315                                  0                     18275

        Report: Total Solver CPU Time (s)     X (m)     Y (m)     Z (m)
0                                  24.532 -0.105754  0.031774  0.002455
1                                  24.532 -0.106013  0.024868  0.015521
2                                  24.532 -0.106014  0.027605  0.006328
3                                  24.532 -0.106013  0.024352  0.008947
4                                  24.532 -0.106007  0.026172  0.016868
..                                    ...       ...       ...       ...
132311                            607.436 -0.117250  0.040575  0.014844
132312                            607.436 -0.117254  0.040656  0.010430
132313                            607.436 -0.117258  0.031591  0.021938
132314                            607.436 -0.117265  0.035741  0.021986
132315                            607.436 -0.117269  0.025780  0.004942

[132316 rows x 16 columns]
```

3.2 2.2 Splitting the data

[6]:
```
X=Granular_flow_data.drop(['Parcel Centroid[X] (m)', 'Parcel Centroid[Y] (m)',␣
↪'Parcel Centroid[Z] (m)', 'X (m)',        'Y (m)',        'Z (m)'], axis=1)
y= Granular_flow_data[['Parcel Centroid[X] (m)', 'Parcel Centroid[Y] (m)',␣
↪'Parcel Centroid[Z] (m)']]
```

[7]:
```
X
```

[7]:

	Unnamed: 0	Time (s)	Particle Velocity[i] (m/s) \
0	0	0.02	0.761367
1	1	0.02	0.648313
2	2	0.02	0.647975
3	3	0.02	0.648059
4	4	0.02	0.648702
...	...	...	...
132311	18270	0.22	0.538821
132312	18271	0.22	0.538780
132313	18272	0.22	0.538739
132314	18273	0.22	0.538658
132315	18274	0.22	0.538617

	Particle Velocity[j] (m/s)	Particle Velocity[k] (m/s) \
0	3.673675e-03	-0.138890
1	2.494714e-04	-0.251882
2	-1.013336e-04	-0.251266
3	4.421311e-05	-0.251678
4	7.440611e-04	-0.250163
...	...	...
132311	1.922194e-07	-0.108071
132312	3.806327e-09	-0.108018
132313	-2.009805e-07	-0.107967
132314	9.289315e-08	-0.107863
132315	-2.137597e-08	-0.107811

	Particle Velocity: Magnitude (m/s)	Volume Fraction of Phase 1 \
0	0.773941	0.064024
1	0.695525	0.077178
2	0.694986	0.064024
3	0.695214	0.064024
4	0.695267	0.077178
...	...	...
132311	0.549552	0.040602
132312	0.549502	0.086906
132313	0.549452	0.048327
132314	0.549351	0.040602
132315	0.549301	0.160566

	Wall Shear Stress: Magnitude (Pa)	Report: Element Count 1 \
0	0	2777
1	0	2777
2	0	2777
3	0	2777
4	0	2777
...	...	...
132311	0	18275

```
132312                                  0                    18275
132313                                  0                    18275
132314                                  0                    18275
132315                                  0                    18275

        Report: Total Solver CPU Time (s)
0                                  24.532
1                                  24.532
2                                  24.532
3                                  24.532
4                                  24.532
..                                   ..
132311                            607.436
132312                            607.436
132313                            607.436
132314                            607.436
132315                            607.436

[132316 rows x 10 columns]
```

[8]:
```
y
```

[8]:
```
        Parcel Centroid[X] (m)  Parcel Centroid[Y] (m)  Parcel Centroid[Z] (m)
0                    -0.105754                0.031774                0.002455
1                    -0.106013                0.024868                0.015521
2                    -0.106014                0.027605                0.006328
3                    -0.106013                0.024352                0.008947
4                    -0.106007                0.026172                0.016868
..                       ..                     ..                      ..
132311               -0.117250                0.040575                0.014844
132312               -0.117254                0.040656                0.010430
132313               -0.117258                0.031591                0.021938
132314               -0.117265                0.035741                0.021986
132315               -0.117269                0.025780                0.004942

[132316 rows x 3 columns]
```

3.3 2.3 Checking if there is any missing value in our data

```
* If there is any non-numerical value, we perform feature encoding
* And/ or if there is any missing value, we perform imputation
* If no missing value, then we continue to building the machine Learning Model
```

[9]:
```
X.isna().sum()
```

[9]:
```
Unnamed: 0                          0
Time (s)                            0
Particle Velocity[i] (m/s)          0
```

```
Particle Velocity[j] (m/s)            0
Particle Velocity[k] (m/s)            0
Particle Velocity: Magnitude (m/s)    0
Volume Fraction of Phase 1            0
Wall Shear Stress: Magnitude (Pa)     0
Report: Element Count 1               0
Report: Total Solver CPU Time (s)     0
dtype: int64
```

[10]:
```
y.isna().sum()
```

[10]:
```
Parcel Centroid[X] (m)    0
Parcel Centroid[Y] (m)    0
Parcel Centroid[Z] (m)    0
dtype: int64
```

4 3.0 Building and Training the Machine Learning Model for our Data

[11]:
```
from sklearn.ensemble import RandomForestRegressor

model1 =RandomForestRegressor(n_estimators=10)
model1.get_params
```

[11]:
```
<bound method BaseEstimator.get_params of
RandomForestRegressor(n_estimators=10)>
```

[12]:
```
np.random.seed(42)
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test= train_test_split(X,y,
                                                   test_size=0.2)

model1.fit(X_train, y_train)
```

[12]:
```
RandomForestRegressor(n_estimators=10)
```

4.1 3.1 Predicting with our model

[13]:
```
y_preds=model1.predict(X_test)

y_preds
```

[13]:
```
array([[-0.07033733,  0.03015237,  0.00623679],
       [-0.09819082,  0.0354989 ,  0.00903425],
       [-0.10166821,  0.03635914,  0.00661332],
       ...,
```

```
       [-0.10083132,  0.0295711 ,  0.01283442],
       [-0.06159568,  0.02427658,  0.00175479],
       [-0.09550467,  0.03003019,  0.01164979]])
```

[14]:
```
y_preds=pd.DataFrame(y_preds, columns=['Parcel Centroid[X] (m)',        'Parcel␣
↪Centroid[Y] (m)',        'Parcel Centroid[Z] (m)'])
y_preds
```

[14]:

	Parcel Centroid[X] (m)	Parcel Centroid[Y] (m)	Parcel Centroid[Z] (m)
0	-0.070337	0.030152	0.006237
1	-0.098191	0.035499	0.009034
2	-0.101668	0.036359	0.006613
3	-0.099405	0.035691	0.003851
4	-0.078971	0.032328	0.006506
...	...	...	...
26459	-0.083500	0.035491	0.003956
26460	-0.052765	0.035463	0.005692
26461	-0.100831	0.029571	0.012834
26462	-0.061596	0.024277	0.001755
26463	-0.095505	0.030030	0.011650

[26464 rows x 3 columns]

[15]:
```
y_test
```

[15]:

	Parcel Centroid[X] (m)	Parcel Centroid[Y] (m)	Parcel Centroid[Z] (m)
86244	-0.071601	0.029988	0.005125
76220	-0.100030	0.036615	0.007536
23934	-0.102130	0.037137	0.006242
91842	-0.099844	0.031683	0.003446
30939	-0.080180	0.032091	0.006331
...	...	...	...
123424	-0.082554	0.039152	0.005637
100623	-0.052231	0.034208	0.005803
92603	-0.100923	0.033062	0.010606
99994	-0.056339	0.022383	0.003182
34119	-0.095481	0.033089	0.011253

[26464 rows x 3 columns]

4.2 3.2 Scoring our model

[16]:
```
model1.score(X_train, y_train)
```

[16]: 0.9772600438344501

[17]:
```
model1.score(X_test, y_test)
```

[17]: 0.8689837633149927

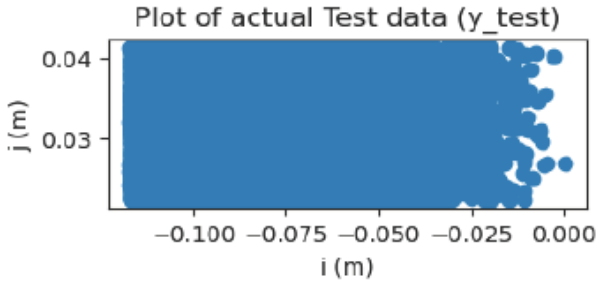
5 4.0 Plotting the Data

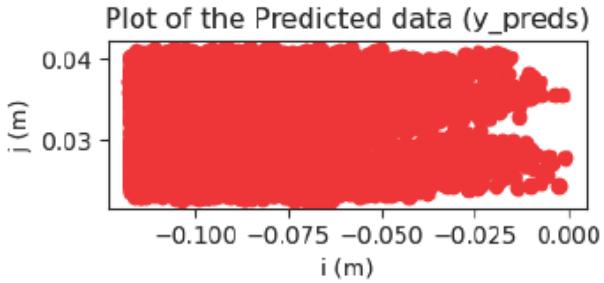
Plotting at the first time-index of the test data This is equivalent to the zeroth index We can do the same for other indexes

[18]:
```
actual_Xi = y_test.iloc[:, :1]
actual_Xj=y_test.iloc[:, :2].drop('Parcel Centroid[X] (m)', axis =1)

predicted_Xi = y_preds.iloc[:, :1]
predicted_Xj = y_preds.iloc[:, :2].drop('Parcel Centroid[X] (m)', axis =1)
```

5.1 4.2 Separate plots of the Actual Granules Position and the Predicted Granules Position

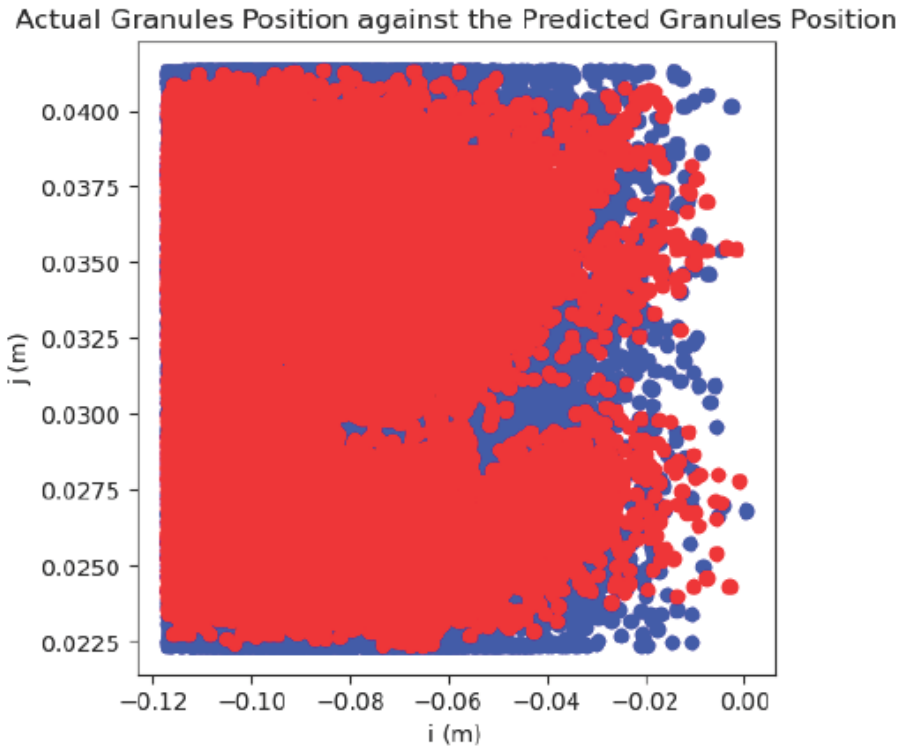
[19]:
```
#Plotting the actual and Predicted positions
# Plot the actual and predicted positions
#plt.scatter(actual_Xi, actual_Xj, color='blue', label='Actual')

fig, (ax1, ax2) = plt.subplots(2,1, figsize=(5,5))

ax1.scatter(actual_Xi, actual_Xj)
ax2.scatter(predicted_Xi, predicted_Xj, color='red')

ax1.set(title="Plot of actual Test data (y_test)",
              xlabel='i (m)',
              ylabel='j (m)'
             )
fig.tight_layout(pad=5)

ax2.set(title= "Plot of the Predicted data (y_preds)",
        xlabel= 'i (m)',
        ylabel='j (m)'
       )

plt.savefig("./Separate plots of Actual and Predicted granules Position.png")
```





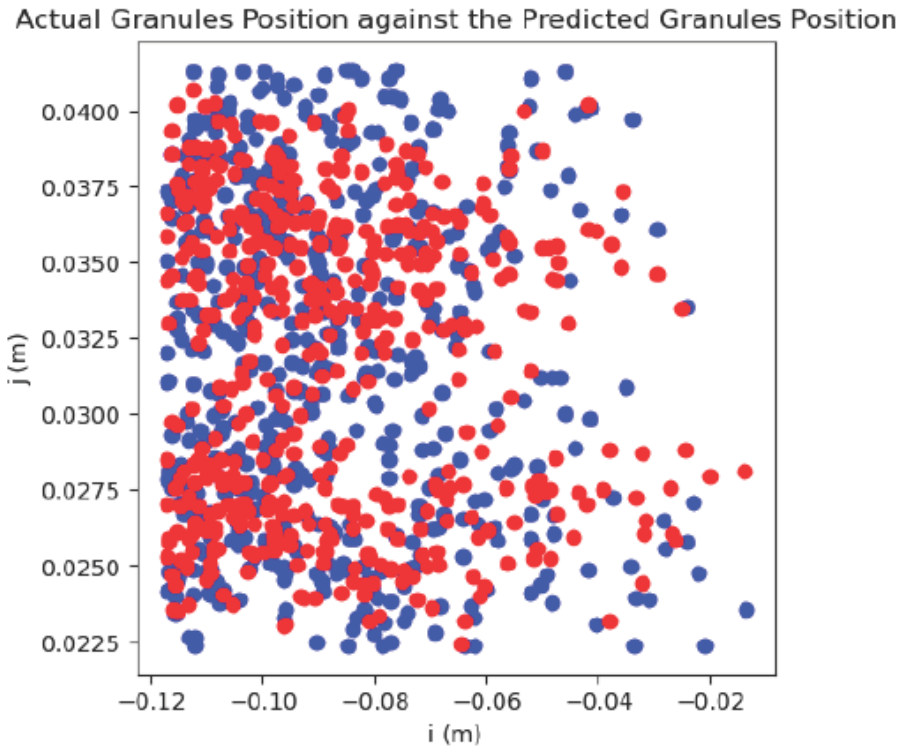
5.2 4.3 Plotting the Actual Granules Position against the Predicted Granules Position

[20]:
```
fig, ax3 =plt.subplots(1,1,figsize=(5,5))

ax3.scatter(actual_Xi, actual_Xj, color ='blue')

ax3.scatter(predicted_Xi, predicted_Xj, color='red')

ax3.set(title="Actual Granules Position against the Predicted Granules␣
 ↪Position",
      xlabel="i (m)",
      ylabel="j (m)")

plt.savefig('./Actual vs Predicted.png')
```



[21]:
```
## 4.3 Plot of the Actual against the Predicted Granules position of the first␣
↪500 granules
```

[22]:
```
fig, ax4 =plt.subplots(1,1,figsize=(5,5))
plt.scatter(actual_Xi[:500], actual_Xj[:500], color ='blue')
plt.scatter(predicted_Xi[:500], predicted_Xj[:500], color='red')

ax4.set(title="Actual Granules Position against the Predicted Granules␣
↪Position",
      xlabel="i (m)",
      ylabel="j (m)")

plt.savefig('./Actual vs Predicted.png')
```



6 5.0 Using Linear Regression Model

[23]:
```
from sklearn.linear_model import LinearRegression

model2=LinearRegression()

model2.get_params
model2.fit(X_train, y_train)
```

[23]:
```
LinearRegression()
```

6.1 5.1 Making Prediction

[24]:
```
yLinear_Preds=model2.predict(X_test)
yLinear_Preds
```

[24]:
```
array([[-0.06841872,  0.03157054,  0.00475152],
       [-0.10180754,  0.03137598,  0.00955407],
```

```
       [-0.10221444,  0.03387586,  0.00753561],
       …,
       [-0.0998795 ,  0.03198919,  0.01344941],
       [-0.05914113,  0.03355112,  0.00375445],
       [-0.09488411,  0.03193539,  0.01299996]])
```

[25]:
```
yLinear_Preds=pd.DataFrame(yLinear_Preds, columns=['Parcel Centroid[X] (m)',␣
 ↪'Parcel Centroid[Y] (m)', 'Parcel Centroid[Z] (m)'])
yLinear_Preds
```

[25]:

	Parcel Centroid[X] (m)	Parcel Centroid[Y] (m)	Parcel Centroid[Z] (m)
0	-0.068419	0.031571	0.004752
1	-0.101808	0.031376	0.009554
2	-0.102214	0.033876	0.007536
3	-0.099374	0.032280	0.004200
4	-0.076805	0.032816	0.004762
…	…	…	…
26459	-0.079196	0.031548	0.004791
26460	-0.053974	0.031127	0.003493
26461	-0.099879	0.031989	0.013449
26462	-0.059141	0.033551	0.003754
26463	-0.094884	0.031935	0.013000

[26464 rows x 3 columns]

[26]:
```
y_test
```

[26]:

	Parcel Centroid[X] (m)	Parcel Centroid[Y] (m)	Parcel Centroid[Z] (m)
86244	-0.071601	0.029988	0.005125
76220	-0.100030	0.036615	0.007536
23934	-0.102130	0.037137	0.006242
91842	-0.099844	0.031683	0.003446
30939	-0.080180	0.032091	0.006331
…	…	…	…
123424	-0.082554	0.039152	0.005637
100623	-0.052231	0.034208	0.005803
92603	-0.100923	0.033062	0.010606
99994	-0.056339	0.022383	0.003182
34119	-0.095481	0.033089	0.011253

[26464 rows x 3 columns]

6.2 5.2 Scoring the Linear Regression based model

[27]:
```
model2.score(X_train, y_train)
```

[27]: 0.5488745530298805

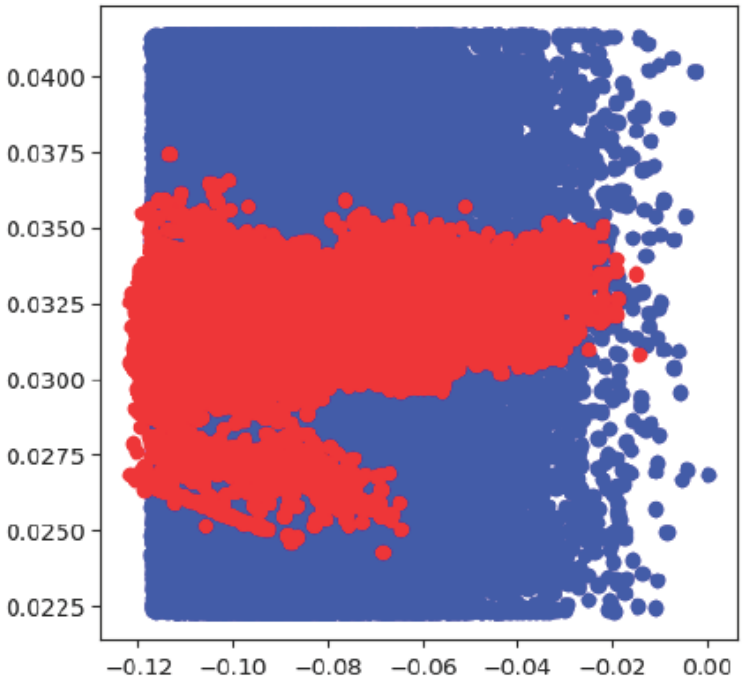
[28]:
```
model2.score(X_test, y_test)
```

[28]: 0.5443689403856448

6.3 5.3 Plots of the Actual vs Predicted Granule Position Using the Linear Regression Model

[29]:
```
LR_Pred_Xi=yLinear_Preds.iloc[:,:1]
LR_Pred_Xj=yLinear_Preds.iloc[:, :2].drop('Parcel Centroid[X] (m)', axis =1)
```

[30]:
```
fig, ax5= plt.subplots(figsize=(5,5))

ax5.scatter(actual_Xi, actual_Xj, color='blue')
ax5.scatter(LR_Pred_Xi, LR_Pred_Xj, color='red')

plt.savefig('./LinearRegression_model.png')
```



7 6.0 Using Ridge Regression

[31]:
```
from sklearn.linear_model import Ridge

model3= Ridge()

model3.get_params

model3.fit(X_train, y_train)
```

[31]:
```
Ridge()
```

[32]:
```
yRidge_preds=model3.predict(X_test)
yRidge_preds
```

[32]:
```
array([[-0.06845366,  0.03159231,  0.00460025],
       [-0.1019072 ,  0.031418  ,  0.00923375],
       [-0.10236349,  0.03384299,  0.00752346],
       …,
       [-0.09964227,  0.03195575,  0.01377098],
       [-0.05929278,  0.03357286,  0.00345722],
       [-0.09454213,  0.0318535 ,  0.01367981]])
```

[33]:
```
yRidge_preds=pd.DataFrame(yRidge_preds, columns=['Parcel Centroid[X] (m)',␣
 ↪'Parcel Centroid[Y] (m)', 'Parcel Centroid[Z] (m)'])
yRidge_preds
```

[33]:

	Parcel Centroid[X] (m)	Parcel Centroid[Y] (m)	Parcel Centroid[Z] (m)
0	-0.068454	0.031592	0.004600
1	-0.101907	0.031418	0.009234
2	-0.102363	0.033843	0.007523
3	-0.099220	0.032259	0.004388
4	-0.076840	0.032803	0.004801
…	…	…	…
26459	-0.079271	0.031545	0.004782
26460	-0.053984	0.031137	0.003475
26461	-0.099642	0.031956	0.013771
26462	-0.059293	0.033573	0.003457
26463	-0.094542	0.031854	0.013680

[26464 rows x 3 columns]

[34]:
```
y_test
```

[34]:

	Parcel Centroid[X] (m)	Parcel Centroid[Y] (m)	Parcel Centroid[Z] (m)
86244	-0.071601	0.029988	0.005125
76220	-0.100030	0.036615	0.007536
23934	-0.102130	0.037137	0.006242

```
91842          -0.099844          0.031683          0.003446
30939          -0.080180          0.032091          0.006331
..                  ..                 ..                ..
123424         -0.082554          0.039152          0.005637
100623         -0.052231          0.034208          0.005803
92603          -0.100923          0.033062          0.010606
99994          -0.056339          0.022383          0.003182
34119          -0.095481          0.033089          0.011253

[26464 rows x 3 columns]
```

[35]:
```
model3.score(X_train, y_train)
```

[35]: 0.5480483619019313

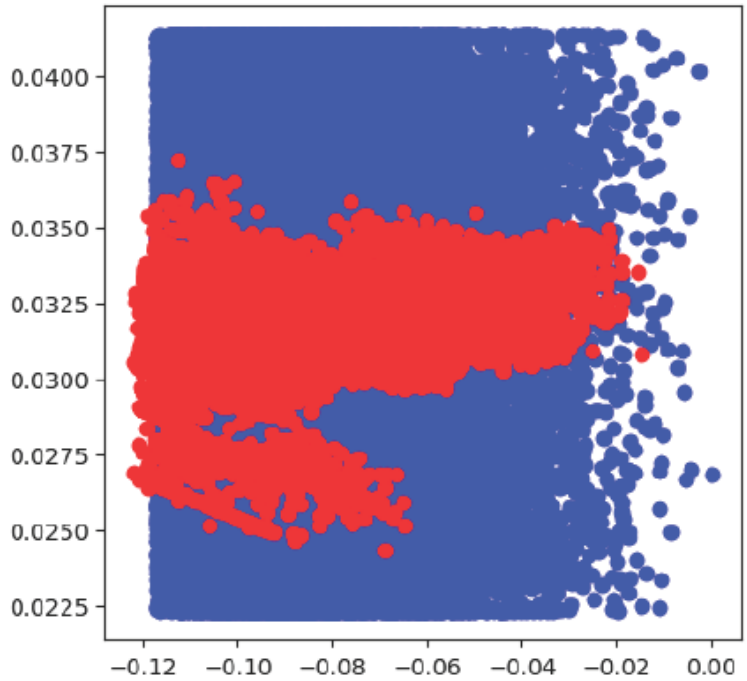
[36]:
```
model3.score(X_test, y_test)
```

[36]: 0.5437974654789722

7.1 6.3 Plotting the graph of Actual against Predicted for the Ridge regression Model

[37]:
```
RR_preds_Xi= yRidge_preds.iloc[:,:1]
RR_preds_Xj= yRidge_preds.iloc[:, :2].drop('Parcel Centroid[X] (m)', axis =1)
```

[38]:
```
fig, ax6= plt.subplots(figsize=(5,5))

ax6.scatter(actual_Xi, actual_Xj, color='blue')
ax6.scatter(RR_preds_Xi, RR_preds_Xj, color='red')

plt.savefig('./Ridge Model plot.png')
```



8 7 Using SVR

[39]:
```
from sklearn.svm import SVR

model4= SVR(kernel="poly", C=100, gamma="auto", degree=3, epsilon=0.1, coef0=1)

model4.get_params

model4.fit(X_train, y_train.iloc[:, 0]) # can not capture the 2D array of y, it␣
↪can predict for 1D
```

[39]:
```
SVR(C=100, coef0=1, gamma='auto', kernel='poly')
```

[40]:
```
y_train.iloc[:, 0]
```

[40]:
```
79814    -0.113688
15519    -0.111130
109659   -0.100971
46932    -0.096796
61115    -0.098551
```

```
                ..
110268   -0.103024
119879   -0.062998
103694   -0.074700
131932   -0.115980
121958   -0.076546
Name: Parcel Centroid[X] (m), Length: 105852, dtype: float64
```

[41]:
```
y_train.iloc[:, 0].shape
```

[41]:
```
(105852,)
```

[42]:
```
y_train.shape
```

[42]:
```
(105852, 3)
```

[43]:
```
model4.score(X_test, y_test.iloc[:,:1])
```

[43]:
```
-1.9068880574062348
```

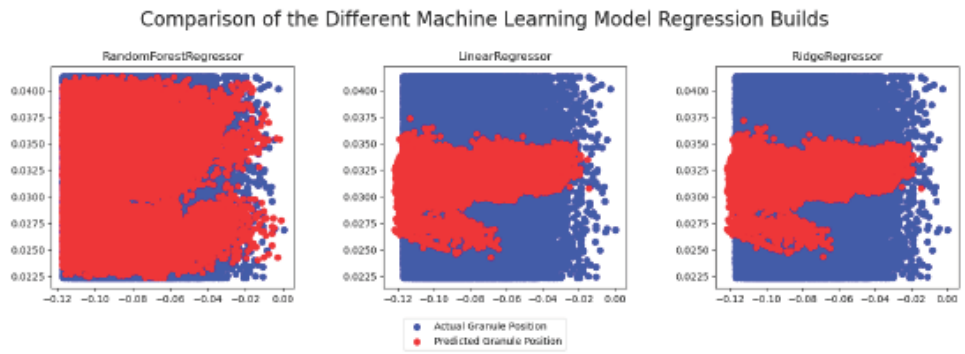
[44]:
```
pd.DataFrame(model4.predict(X_test))
```

[44]:
```
              0
0     -0.056474
1     -0.056474
2     -0.056474
3     -0.056474
4     -0.056474
..          ..
26459 -0.056474
26460 -0.056474
26461 -0.056474
26462 -0.056474
26463 -0.056474

[26464 rows x 1 columns]
```

[45]:
```
y_test.iloc[:,0]
```

[45]:
```
86244    -0.071601
76220    -0.100030
23934    -0.102130
91842    -0.099844
30939    -0.080180
                ..
123424   -0.082554
100623   -0.052231
92603    -0.100923
```

```
99994    -0.056339
34119    -0.095481
Name: Parcel Centroid[X] (m), Length: 26464, dtype: float64
```

9 8.0 Comparing all the Regression Models

[46]:
```
fig, (ax3, ax5, ax6) =plt.subplots(1,3, figsize=(15,5))


l1=ax3.scatter(actual_Xi, actual_Xj, color ='blue')
l2=ax3.scatter(predicted_Xi, predicted_Xj, color='red')
ax3.set(title='RandomForestRegressor')

ax5.scatter(actual_Xi, actual_Xj, color='blue')
ax5.scatter(LR_Pred_Xi, LR_Pred_Xj, color='red')
ax5.set(title='LinearRegressor')

ax6.scatter(actual_Xi, actual_Xj, color='blue')
ax6.scatter(RR_preds_Xi, RR_preds_Xj, color='red')
ax6.set(title='RidgeRegressor')

fig.tight_layout(pad=5.0)
fig.legend(['Actual Granule Position', 'Predicted Granule Position'],␣
 ↪loc='lower center')
fig.suptitle('Comparison of the Different Machine Learning Model Regression␣
 ↪Builds', fontsize=20)
plt.savefig('./Comparison of the Different Machine Learning Model Regression␣
 ↪Builds.png')
```